MODUL PRAKTIKUM 2

ARCHITECTURAL PATTERNS & SYSTEM DESIGN

Pemrograman Sisi Server

Suprayogi,M.Kom

Informasi	Keterangan
Pertemuan	Minggu 2
Sub-CPMK	Menganalisis dan menerapkan architectural pattern yang tepat untuk pengembangan aplikasi backend
Fokus	Arsitektur backend + melanjutkan server Python dari Modul 1
Praktik	Virtual environment, endpoint, modularisasi Python, Git & GitHub

1. Posisi Praktikum dalam RPS

Minggu 2 membahas Architectural Patterns dan System Design. Mahasiswa mengenal monolithic, microservices, event-driven, dan serverless serta memahami bahwa pilihan arsitektur harus didasarkan pada kebutuhan sistem dan trade-off, bukan sekadar karena sebuah teknologi terlihat lebih modern.

Batas Minggu 2
Mahasiswa belum menggunakan Django, ORM, database, atau Docker. Server Python sederhana dipakai sebagai media untuk memahami endpoint, modularitas, dan hubungan antara kode dengan diagram arsitektur. Docker dimulai pada Minggu 3 dan Django pada Minggu 4.

2. Capaian Praktikum

- Menjelaskan monolithic, microservices, event-driven, dan serverless secara sederhana.
- Membandingkan modular monolith dan microservices berdasarkan kebutuhan sistem.
- Mengaktifkan kembali virtual environment dan menjalankan server Python dari Modul 1.
- Menambahkan endpoint HTTP sederhana untuk domain Simple LMS.
- Melakukan refactoring kode ke beberapa module Python berdasarkan tanggung jawab.
- Menggunakan Git dan GitHub sebagai workflow berkelanjutan.
- Membuat diagram arsitektur, module boundary, dan ERD awal Simple LMS.

3. Alokasi Aktivitas

Tahap	Aktivitas	Estimasi
Review Modul 1	venv, server, Git status	15 menit
Konsep	4 pola arsitektur + trade-off	25 menit
Guided Coding 1	Tambah endpoint LMS	30 menit
Guided Coding 2	Refactoring menjadi modules	35 menit
Git Workflow	Commit dan push bertahap	15 menit
Design Exercise	Diagram + ERD	30 menit
Mini Challenge	Tambah Enrollment module	20 menit
Capstone	Milestone 2	20 menit

4. Review Singkat Modul 1

membuat folder baru minggu2, lakukan .

```
mkdir minggu-2

cd minggu-2

python -m venv .venv


# Aktivasi pada Command Prompt:
  .venv\Scripts\activate.bat

# Windows PowerShell
  .\.venv\Scripts\Activate.ps1

python --version


git init

git status
```

5. Konsep Inti Architectural Patterns

5.1 Modular Monolith

```
Client
  |
  v
+---------------------------+
| One Backend Application   |
| Users | Courses | Lessons |
| Assignments | Submissions |
+-------------+-------------+
              |
              v
           Database
```

Satu aplikasi tetap dapat memiliki pemisahan module yang jelas. Untuk Capstone kelas, modular monolith menjadi baseline karena lebih sederhana untuk dikembangkan, diuji, dan dijalankan oleh tim kecil.

5.2 Microservices

```
Client/API Gateway
  |----> User Service
  |----> Course Service
  |----> Assignment Service
```

Service dapat berjalan dan di-deploy secara independen, tetapi kompleksitas komunikasi jaringan, observability, deployment, dan konsistensi data meningkat.

5.3 Event-Driven

```
Payment Service --> Event --> Message Broker
                              |--> Warehouse
                              +--> Notification
```

Komponen berkomunikasi melalui event. Cocok untuk proses asynchronous. Pada Minggu 2 cukup dipahami secara konseptual.

5.4 Serverless

```
Upload Image --> Cloud Trigger --> resize_image() --> Output Images
```

Developer menjalankan fungsi tanpa mengelola server secara langsung. Infrastruktur dikelola penyedia cloud. Minggu 2 tidak mengimplementasikan serverless.

6. Memilih Arsitektur secara Sederhana

Pertanyaan	Pertimbangan
Seberapa besar sistem?	Project latihan di kelas berbeda dengan platform nasional.
Seberapa besar tim?	Tim kecil biasanya membutuhkan operasional lebih sederhana.
Perlu deployment independen?	Jika tidak, microservices mungkin belum diperlukan.
Ada proses asynchronous?	Event-driven dapat relevan untuk notifikasi/proses latar.
Mampu mengelola kompleksitas?	Arsitektur membawa biaya operasional dan debugging.
Prinsip Arsitektur yang baik adalah arsitektur yang sesuai kebutuhan. Lebih kompleks tidak selalu lebih baik.	

7. GUIDED CODING LAB 1 — Menambah Endpoint Simple LMS

Mulai dari server Python sederhana di Modul 1. Kita akan membuat beberapa endpoint agar mahasiswa melihat bahwa satu backend dapat melayani beberapa fungsi bisnis.

```
GET /
GET /health
GET /courses
```

```
GET /students
GET /assignments
```

Contoh implementasi awal satu file:

```
minggu-2\
```

```
# server.py
from http.server import BaseHTTPRequestHandler, HTTPServer
import json

class SimpleHandler(BaseHTTPRequestHandler):
    def send_json(self, data, status=200):
        body = json.dumps(data).encode("utf-8")
        self.send_response(status)
        self.send_header("Content-Type", "application/json")
        self.end_headers()
        self.wfile.write(body)

    def do_GET(self):
        if self.path == "/":
            self.send_json({"message": "Simple LMS Backend"})
        elif self.path == "/health":
            self.send_json({"status": "ok"})
        elif self.path == "/courses":
            self.send_json({"courses": [
                {"id": 1, "name": "Pemrograman Sisi Server"},
                {"id": 2, "name": "Basis Data"}
            ]})
        elif self.path == "/students":
            self.send_json({"students": [
                {"id": 1, "name": "Andi"},
                {"id": 2, "name": "Siti"}
            ]})
        elif self.path == "/assignments":
            self.send_json({"assignments": [
                {"id": 1, "title": "Backend Fundamentals"}
            ]})
        else:
            self.send_json({"detail": "Not Found"}, 404)

server = HTTPServer(("localhost", 8000), SimpleHandler)
print("Server running at http://localhost:8000")
server.serve_forever()
```

Uji melalui browser atau tool HTTP yang tersedia:

```
http://localhost:8000/
http://localhost:8000/health
http://localhost:8000/courses
http://localhost:8000/students
http://localhost:8000/assignments
```

Checkpoint 2
Tunjukkan minimal empat response JSON yang berhasil dan satu response 404. Jelaskan request, endpoint, handler, dan response.

```
git add .
git commit -m "feat: add LMS endpoints"
```

```
git remote add origin https://github.com/USERNAME/pss-minggu-2.git
```

```
git remote -v
```

```
git branch
```

```
git branch -M main
```

```
git push -u origin main

git log --oneline -1
```

8. Jika Semua Kode di server.py

- File akan semakin panjang ketika domain bertambah.
- Data dan logic berbagai domain bercampur.
- Sulit menentukan tanggung jawab setiap bagian.
- Perubahan satu fitur lebih mudah memengaruhi bagian lain.
- Kolaborasi tim menjadi lebih sulit.

Masalah ini menjadi jembatan menuju konsep module boundary dan modular monolith.

9. GUIDED CODING LAB 2 — Refactoring menjadi Modules

Ubah struktur project menjadi:

```
minggu-2/
├── server.py
├── modules/
│   ├── __init__.py
│   ├── courses.py
│   ├── students.py
│   └── assignments.py
├── .venv/
├── .gitignore
└── README.md
```

Isi module Courses:

```
# modules/courses.py
courses = [
    {"id": 1, "name": "Pemrograman Sisi Server"},
    {"id": 2, "name": "Basis Data"},
]

def get_courses():
    return courses
```

Module lain menggunakan pola yang sama:

```
# modules/students.py
students = [
    {"id": 1, "name": "Andi"},
    {"id": 2, "name": "Siti"},
]

def get_students():
    return students
```

```
# modules/assignments.py
assignments = [
    {"id": 1, "title": "Backend Fundamentals"}
]

def get_assignments():
    return assignments
```

Kemudian gunakan module tersebut dari server:

```
from modules.courses import get_courses
from modules.students import get_students
from modules.assignments import get_assignments
```

```
# di dalam do_GET()
elif self.path == "/courses":
    self.send_json({"courses": get_courses()})
elif self.path == "/students":
    self.send_json({"students": get_students()})
elif self.path == "/assignments":
    self.send_json({"assignments": get_assignments()})
```

Konsep yang harus dipahami
Aplikasi masih satu server dan satu proses. Kita tidak membuat microservices. Yang berubah adalah organisasi internal kode berdasarkan tanggung jawab: inilah ide dasar modular monolith.

10. Git & GitHub sebagai Workflow Berkelanjutan

Jangan menunggu akhir praktikum untuk membuat satu commit besar. Buat commit setelah perubahan logis.

```
# setelah refactoring
git add .
git commit -m "refactor: organize backend into modules"

# setelah dokumentasi
git add .
git commit -m "docs: add architecture notes"

git push origin main
```

Checkpoint 3
Repository GitHub harus menunjukkan beberapa commit yang menggambarkan evolusi pekerjaan, bukan satu commit besar di akhir.

11. Design Exercise — Dari Kode ke Diagram

Setelah menyelesaikan Guided Coding Lab, perhatikan kembali aplikasi yang telah dibuat. Walaupun program masih sederhana, aplikasi tersebut sebenarnya sudah memiliki struktur arsitektur.

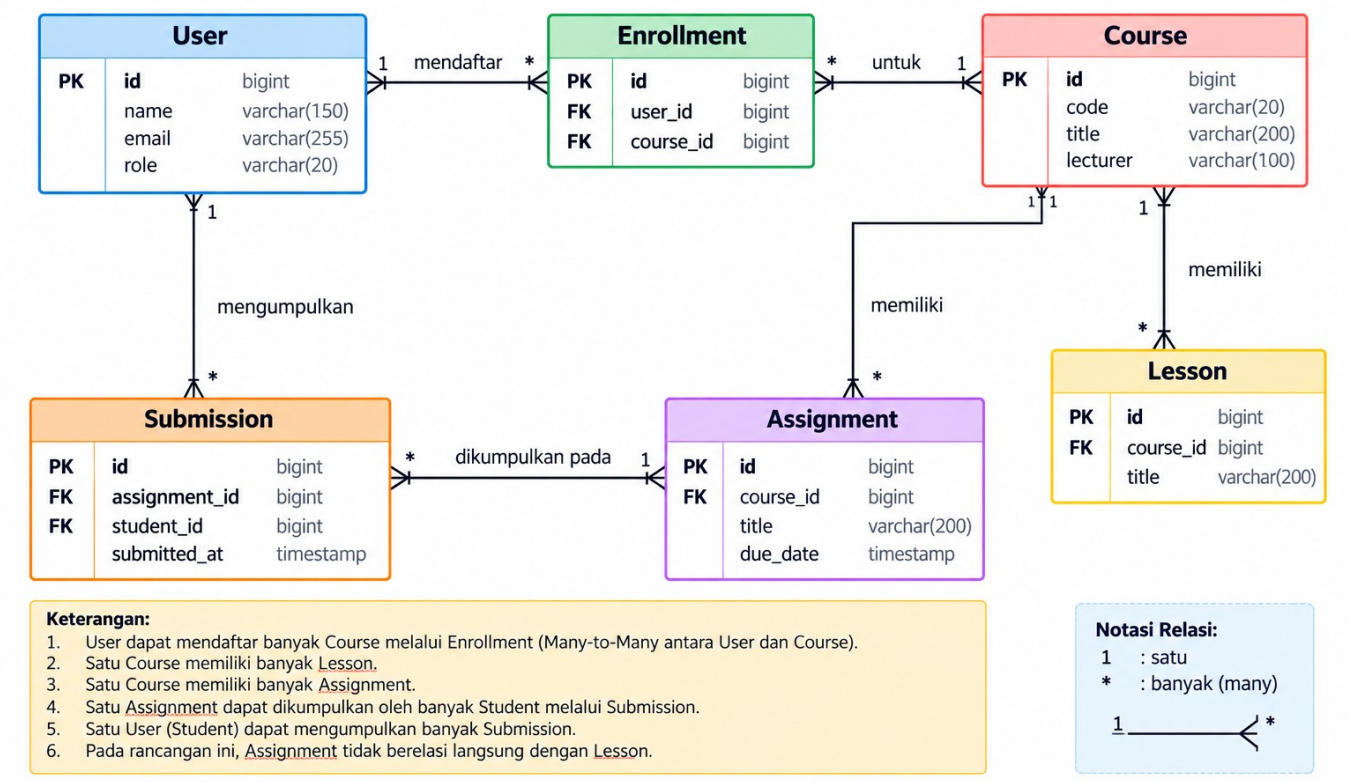
Diagram berikut menunjukkan hubungan antara **client, Python backend, dan modul-modul yang telah dibuat**:

```
+-------------------+
| Browser / Client  |
+---------+---------+
          | HTTP
          v
+---------------------------+
| Python Backend            |
|                           |
| Courses | Students        |
| Assignments               |
+---------------------------+

(Database belum digunakan)
```

12. ERD Awal Simple LMS

Walaupun database belum diimplementasikan, mahasiswa dapat melihat rancangan data yang nantinya dibutuhkan.




```
User 1 ----- * Enrollment * ----- 1 Course
Course 1 ----- * Lesson
Course 1 ----- * Assignment
Assignment 1 - * Submission
User 1 ------- * Submission
```

Entity	Contoh Data Utama
User	id, name, email, role
Course	id, code, title, lecturer
Enrollment	id, user_id, course_id
Lesson	id, course_id, title
Assignment	id, course_id, title, due_date
Submission	id, assignment_id, student_id, submitted_at
Catatan	
ERD Minggu 2 bersifat rancangan awal. Detail field, constraint, migration, dan ORM akan diperdalam pada Modul 4.	

13. MINI CHALLENGE — Tambahkan Enrollment

Tanpa diberikan source code lengkap, tambahkan satu module baru `enrollments.py`.

1. Buat data enrollment sederhana.
2. Buat fungsi `get_enrollments()`.
3. Tambahkan endpoint `GET /enrollments`.
4. Pastikan endpoint lain tetap berjalan.
5. Commit dengan pesan yang menjelaskan perubahan.

Tujuan Mini Challenge
Mahasiswa membuktikan bahwa mereka memahami pola modularisasi, bukan sekadar menyalin contoh Courses/Students/Assignments.